Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 04

Lecture – 20

Lec-20: VR Software

Hello and welcome to the NPTEL MOOC's course on Foundations for Virtual and Augmented Reality Systems. We are going to continue our discussion on VR software; that is, we will continue with lecture number 10, which we started in the last lecture. Now, previously, if you recollect, we talked about the broad idea of VR software. In this context, we have introduced a very abstract way of looking at the VR software system, which is the virtual world generator. which takes as input from different input devices and produces output that is supposed to stimulate our sensory organs to give us an immersive experience.

Now, the virtual world generator is a very abstract concept. Just like a black box, we are discussing what kind of things should go inside the black box, inside the virtual world generator concept. In line with this discussion, we have introduced desirable components of the virtual world generator. We started our discussion with the support required for synthetic virtual world generation, that is, support for computer graphics operations.

Also, we talked about how realistic virtual world generation primarily requires support for computer vision or a mixture of both using texture synthesis, which again requires computer graphics support. Also, we talked about support required for matched motion and locomotion concepts. Now, we are continuing our discussion, and let us see what other key components should be part of the virtual world generator abstraction. Another key support that is required is physics support.

So along with computer graphics for rendering, computer vision again for rendering, support for locomotion and matched motion, we also need to have support for physics. Now, the virtual world generator usually implements some physics. It depends on the kind of generator we are using and the extent of support that they can provide. The basic idea is

to make the virtual world appear to behave like the real world. What do we get to see around us?

Similar things should happen in the virtual world. So, this support is required. In most cases, the basic laws of mechanics should govern how objects move in the virtual world. So, support for basic laws should be good enough. Advanced laws support makes things more realistic, which sometimes may be desirable. For example, if you drop an object, what kind of things are you likely to see? Due to gravity, it should accelerate toward the ground. So that type of trajectory, that type of movement, should be supported by the virtual world generator.

So, when we are trying to create some animation involving the dropping of a ball, the ball should not move toward the ground with a constant motion. Ideally, it should accelerate due to gravitational force. Another important component is collision, the collision of one object with other objects. So, the collision detection algorithm support ideally should be part of the virtual world generator. So, it determines whether two or more bodies are intersecting in the virtual world, and if they are intersecting, then how they should behave.

Also, only detection is not sufficient; what happens after the collision takes place? If a new collision occurs, an appropriate response is also needed. That should be part of the collision detection algorithm as part of collision response. For example, suppose a user pokes his head through a wall in the virtual world. Now, the question is, should the head in the virtual world be stopped even though it continues to move in the real world? Ideally, when there is a hard surface and we are trying to poke through it, then there is a collision and we are unable to poke through it in the real world. So, whether the same thing should happen in the virtual world whereas we know that in the real world there is no such obstacle, no such wall, and our head is moving freely. The physics support should also take into account other aspects of our physical world, such as how light propagates through the environment, how light interacts with different surfaces, what kind of effects they generate, what are the sources of light, and depending on the sources, different types of illumination effects we can see.

How do sound and smell propagate, just like light, and so on. So, these are some of the other things that physics support should provide; that is, the virtual world generator should provide such support for replicating what happens in the real world. Another key component, although it may not be required everywhere. Sometimes we may require the network experience. Sometimes in a virtual world, we can have collaborative interaction.

That is, we can have a collaborative VR experience created over a network. And what is that? A shared virtual world is maintained by a system or a server. Each user has a distinct matched zone. Matched zones may overlap in the real world.

One must then be careful to avoid unwanted collisions, assuming that the users are close to each other. Most often, Such matched zones are disjoint and distributed around the earth. So, if the users are close by, we should be careful when the users are moving in their matched zones; they should not collide with each other. However, in most cases, the users may be spread around the globe, around the earth.

So, there is no possibility of physical collision. Within the virtual world, user interactions, including collisions, must be managed by the VR system whether they are close by or spread around the world. Now, if multiple users are interacting in a social setting, burdens of matched motions may increase; that is, how do I take care of the matched motion? When social interaction is important. As users meet each other, they could expect to see eye movements, facial expressions, and body language.

So, the support that the VWG provides should support these types of activities in the virtual world when we are in a networked collaborative VR setting. So, these are some of the key components that ideally the black box, the virtual world generator, should support, and accordingly we can assume that if we are going to use some software, some tools, or some system for development and deployment, they come with support for these components. Now, let us shift our focus to what choices the developer has in order to develop and deploy a system. So, the developer can start with a basic SDK, software development kit, for a specific headset or device or platform.

We have already talked about that before. Build their own virtual world generator from scratch using the SDK support for that specific hardware or platform. Now, in such cases, the SDKs or the development kits should provide basic drivers to drive the hardware, interfaces or APIs to access tracking data, and APIs to make calls to graphical rendering libraries. That is typically what SDKs are expected to do.

But these are some of the components. So, when we say that basic drivers and trackers are there, then it takes care of some aspects of a VWG. Rendering takes care of some other aspects of a VWG, but higher level aspects are not taken care of. For example, locomotion is partly taken care of by tracking, but not Not every aspect of locomotion or matched motion is taken care of by tracking.

Then there are concerns related to physics, related to network experience. Those are not typically taken care of by SDKs. So, the developer must build the physics, at the very least, of the virtual world from scratch, Handling problems such as movement of an avatar and corresponding collision, occurrence of collision and the response, collision detection, lighting models, audio support, and so on. So, here note that we have ignored the other aspect that is networked collaborative VR experience. The minimum thing that is required is physics support. Sometimes we may require collaborative VR experience support as well. So if such an approach is adopted by the developer, then the developer gets maximum control and ability to optimize performance because The physics support the algorithms that the developer can choose and implement depending on their performance.

rather than being forced to use whatever comes with a package. But what is the cost? The cost is a difficult implementation burden that the developer has to take a lot of pain. Has to expend a lot of effort in understanding and implementing the overall system by developing things from scratch. Alternatively, as we have seen earlier, we have discussed earlier, the other choice that the developer can exercise is the developer may use a ready-made virtual world generator.

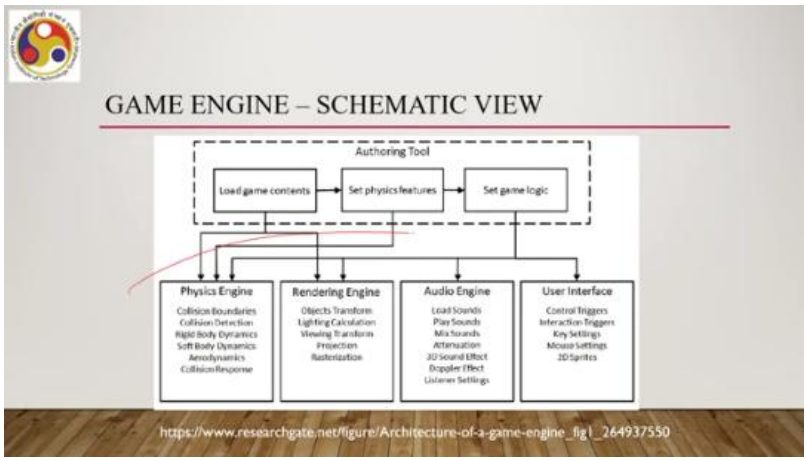
Customized to make particular experiences by choosing menu options and writing high-level scripts. So there can be some ready-made virtual world generator which the developer can use and perform some customization. For example, OpenSimulator or Unity3D can provide such ready-made VWG. OpenSimulator was designed as an open source alternative to Second Life, a particular experience for building a virtual society of avatars. Second Life once upon a time used to be a very popular game to build alternative reality. And OpenSimulator was designed to support a developer in doing that. Unity3D is a game engine adapted to work for virtual reality.

which is one of the popular game engines. And we will... probably use the term Unity3D or the reference to this Unity3D more than OpenSimulator in subsequent discussions. The idea is that such higher-level engines make it very easy for developers to make a VR experience in little time because they come with all the supports including support for collisions or physics. The developer does not have to spend time and effort building things from scratch. However, what is the drawback? The drawback is that it is harder to make highly original experiences not imagined by the engine builder. So, such engines or such readymade VWGs come with their own set of function supports. Own set of algorithm implementations, which the developer must use to create the experience.

In that process, the developer is restricted by the system. Developer cannot do things originally, unlike the previous approach, but it reduces the effort. So either of these two choices a developer can exercise to build the experience. Now we know the basic idea of VR software from the developer's point of view. What are the components and what are the things that the developer can use?

What are the choices that the developer has to go ahead and build a system? Those are discussed at a very abstract level. Now, let us try to get some idea of the practical systems, actual systems that are in use in this domain of VR experience development. So, we will start our discussion with the concept that we have repeatedly referred to that is VR engine or game engine. Now, a game engine is a middleware.

It sits between the hardware and the software functionalities, higher level or application level software functionalities, and it. Abstracts the hardware functionalities for the developers of XR applications. So, it comes with some support for hardware abstractions. It is a software development environment to build XR experience and it can be considered to be a readymade VWG on top of which a developer can build their own application. Schematically, we can think of a game engine as comprising a few things.



comprising multiple engines, say for example, physics engine, rendering engine, audio engine and user interface. Now, there can be an authoring tool which the developer can use and through that authoring tool can access the different functionalities supported by the game engine. To develop their own software which runs on top of the game engine. So the authoring tool may provide loading options, various feature setting options and logic setting options. Sometimes this logic setting can be done through scripting, which can be part of a game engine development environment.

In fact, the entire authoring tool can be part of a development environment comprising the game engine plus the other components. An example is Unity 3D Development Environment or Unity Hub. So then we can say that typically game engines have some core functionalities which are rendering support, physics support, audio support, scripting for customizing various features. networking support primarily for multiplayer collaborative VR experience as well as some low level support for various platform specific tasks including memory management and other activities. Threading, localization support, these are all part of memory management.

There can be many other basic level operations that are supported by various game engines. So, let us try to understand what these individual engines mean. Say for example, when we are talking about rendering engine, what is this engine supposed to do? It is supposed to help generate animated 3D graphics by any of a number of methods.

So, there are different rendering techniques such as rasterization or ray tracing. A particular rendering engine may support a particular rendering method. Rasterization is less computationally intensive as compared to the ray tracing method. Depending on the sophistication level of the engine or the platform, different rendering methods may be adopted. Now, typically such rendering engines support being built upon one or multiple rendering APIs such as Direct3D, OpenGL, or Vulkan, which provide software abstractions of the GPU. The GPU is the hardware component that is used for speeding up the rendering process. So, we can roughly say that if we create a stack of different layers, then the lowest layer is the hardware. On top of it lies these rendering APIs that can be considered another layer, such as OpenGL. On top of that, we can have the game engine, and on top of that, we can have the application software.

which accesses the game engine; the application software accesses the game engine, the game engine accesses these layers, and this layer accesses the hardware. So, in that way, we can think of different layers of a typical VR system when we are assuming a middleware, such as a game engine, is being used. The hardware components here include the GPU. as well as other hardware components such as the CPU and memory. The audio engine is another part of the basic game engine support.

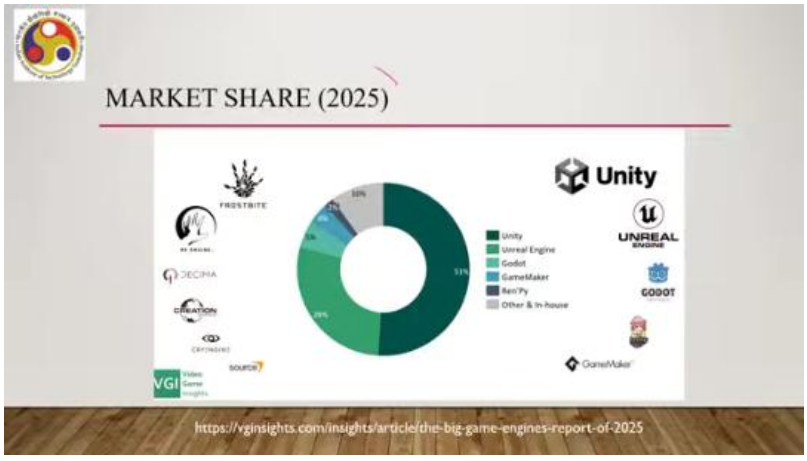
It consists of algorithms related to loading, modifying, and generating sound through a speaker system. It can perform calculations on a CPU. So, no GPU support is required. or on a dedicated ASIC application-specific integrated circuits. So, typically when we are

talking about audio engines, we do not associate them with any GPU, which is generally associated with graphics or rendering.

Now, like in the case of a rendering engine, in the case of an audio engine also, they can be built on top of abstraction APIs such as OpenAL, SDL audio, X audio 2, Wave audio, etc. That is also possible. So, again we can think of a layered architecture here. Physics engine, one of the crucial components of any game engine. These are responsible for emulating the laws of physics as we have already discussed realistically within the application.

So, typically they provide a set of functions for simulating physical forces and collisions acting on various objects within the scene at runtime. So, we have already touched upon this issue earlier. So, these are the basic components that are often part of any game engine. Now, there are actually many game engines are available, not all contain all these components. It depends on which game engine we are using. So, as I said, there are many game engines. For example, Unity 3D, Unreal Engine 4, CryEngine, Godot, Lumberyard, Amazon Sumerian, and so on. So, all of these—Unity 3D, Unreal, CryEngine, Godot,

Lumberyard, Amazon Sumerian—all of these can be considered examples of game engines because they support the basic characteristics. However, if we look at the market share, the latest market share, then we can see that Unity is occupying almost 51% of the game engine market share. So, we can say that that is followed by Unreal Engine. 28%, and all others put together comprise the rest. So, we can say that Unity 3D is quite popular.



Unreal Engine is also quite popular. Unreal Engine's major features are freely available, whereas Unity Engine is proprietary. It has a free version, but that comes with limitations;

proprietary versions give more flexibility. So the most popular engines are Unity 3D and Unreal Engine based on the latest statistics.

That does not mean that other engines are bad compared to these two. It means that people are using those engines more. Probably they are better supported compared to the other ones. That is about the basic idea of game engines that are being used in practice for building VR applications. Now let us shift our focus to another aspect.

So once you develop it, you need to deploy it on some device. So what are the tools? What are the packages or software that the developer can make use of for deployment of their application? When we are talking about deployment, we should talk about different types of deployment.

So, these types can be broadly of two types. One is native application, which requires use of SDKs tailored to the platform and the hardware. So, when we are talking about platform, typically we refer to the OS plus the hardware, Windows or Mac. for desktop or Android and iOS for mobile. So, Windows or Mac OS is typically referred to for desktop whereas Android and iOS refer to mobile platforms.

Now, when we are trying to deploy, we should know what the target deployment type is, whether it is a native application or it is a web application. Web means the experience can be obtained remotely; no need to use any specific device. So, let us first get some idea on the native deployment SDKs. So, we know that SDKs are software development kits; already we have talked about those.

They are nothing but a collection of software development tools in one installable package specific to hardware and OS combination. And they provide fundamental tools to design, build, and test VR experiences for specific platforms. Now, these SDKs. for native deployment can be categorized in two ways based on type. So, we have two ways of categorization: one is based on type whether we are talking about VR applications or AR applications and secondly, based on availability, that is whether they are proprietary or open source. So in either of these two ways, we can categorize the native SDKs. Now, since we are discussing VR software, let us shift our focus to VR SDKs or SDKs used for VR application development. Now, HMDs or head-mounted displays are the most popular device types for VR experiences. So, for different HMDs, there are different SDKs, as we noted earlier.

For HTC Vive system, we have OpenVR SDK by Valve or SteamVR SDK or VRTK. Any of these can be used. VRTK is quite old. For Oculus Rift, we have the Oculus SDK. However, the Oculus Rift is no longer supported or very popular.

We have Oculus Quest. Then for Samsung Gear VR, we have the Oculus Mobile SDK; Sony PlayStation, we have the PSVR SDK; and there is Google Daydream View or Google Cardboard. which is supported by the Google VR SDK. Now, the Google VR SDK is open source, but it also is no longer very much supported.

Those are some of the popular SDKs still in use or that were quite popular even in the recent past. Now, let us learn something about web deployment: how we can deploy our systems over the web. Now, when we are talking about web deployment, we have access to several tools. These tools can be used for web deployment. Now, these tools can be categorized, like native SDKs.

There can be some broad categorization of such tools. For example, some tools are there which are good for cloud hosting and rendering over a network. They only provide hosting and rendering support. Some tools are nothing but libraries for development support, development for deployment over the web. Then we can have tools for complete development and deployment of applications on the web.

There can be some generic development editors which can be utilized for writing code for web-based deployment, for example, the Glitch editor. And there can be SDK support as well, but primarily meant for web deployment such as AR Core. What are the cloud hosting and rendering support tools that are there? Some examples are Azure Remote Rendering for 3D rendering, NVIDIA CloudXR.

Immersive Stream for XR, which works on Google Cloud. Eco3D and Arbor XR. Then libraries for development. Under this category, we can have WebXR, which uses WebGL. Three.js makes WebGL easier, used for rendering.

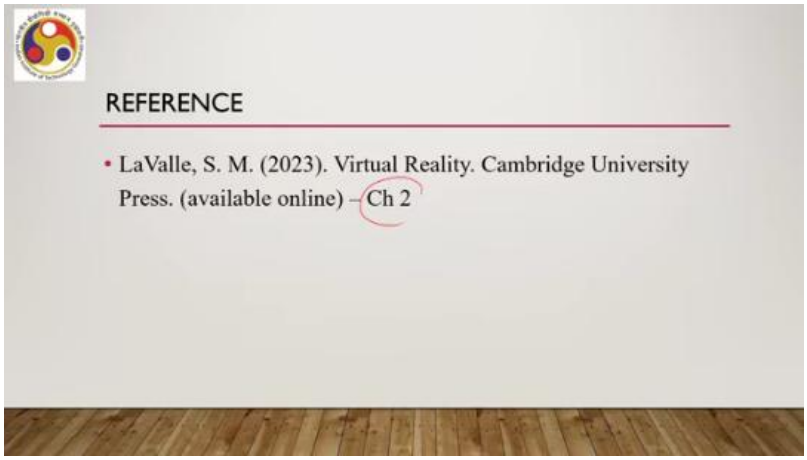
AR.js three.xr.js is like WaveXR built on three.js. AFrame, Babylon.js, ReactXR.js. In fact, there are many more libraries. These are some of the popular ones.

Then we can also have complete deployment and development tools such as ZappAir, 8thWall, MyWebAR, VRDirect, GmetriXR, InstaVR, PlugXR, Amazon Sumerian, and so on. So, these are broadly the categories of web deployment tools. The editor and SDKs, we have already seen the examples. We conclude our discussion on VR software.

So before we finish, let us quickly summarize what we have covered. We got a broad idea of VR software. We have seen or tried to understand the software in terms of an abstraction, a virtual world generator, and we have seen the key components that the abstraction should support for a developer to be able to build VR applications. We have also seen developers' choices.

The two choices are there. One is using ready-made VWG or using SDKs and developing things from scratch. So, we have choice number one: using SDKs and developing things from scratch, or choice number two: using ready-made VWGs. Then we talked about VR game engine ideas and the popular engines as of today. And finally, we talked about deployment tools.

What are the tools available for deployment, either on native systems or over the web? Next, we will cover AR software. For reference, Chapter 2 of this book can be used. Some of the material in this lecture is taken from Chapter 2 of this book. So I hope you have enjoyed the content.



So we have finished our discussion on VR software. You understood the concepts, enjoyed it, and are in a position to start using VR software. So next, as we will be covering AR software, I hope you will get more understanding of overall XR software. So let us meet in the next lecture on AR software. Thank you and goodbye.

Happy learning.